SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

192321047 - KAUSHIK

CSA1735 – Artificial Intelligence

CO1 Assessment 2 (AT2)

Constraint-Based Problem Solving

Student: Kaushik Narayanan V

Register No: 192321047

1. Hospital Shift Scheduling (Backtracking Search)

CSP: Variables D1, D2, D3. Shifts: M = Morning, A = Afternoon, N = Night (order M < A < N). Domain of each variable = {M, A, N}.

Constraints: (i) D1 ≠ N; (ii) D2 before D3, i.e. D2 < D3; (iii) AllDiff(D1, D2, D3), one doctor per shift; (iv) D3 ≠ M; (v) every doctor gets exactly one shift.

Pre-pruning (unary and ordering): D1 {M, A}; D3 {A, N}; D2 must be earlier than D3 so D2 cannot be N, giving D2 {M, A}.

Backtracking trace (variable order D1, D2, D3; value order M, A, N)

Step	Try	Constraint checks	Result
1	D1 = M	(i) M ≠ N ✓; AllDiff ✓	Keep
2	D2 = M	AllDiff ✗ (Morning already used by D1)	Reject
3	D2 = A	AllDiff ✓; (ii) not yet checkable	Keep
4	D3 = M	(iv) D3 ≠ M ✗	Reject
5	D3 = A	AllDiff ✗ (Afternoon used	Reject

192321047 K AUSHIK

Step	Try	Constraint checks	Result
		by D2)	
6	D3 = N	(iv) ✓; AllDiff ✓; (ii) A < N ✓; all doctors assigned (v) ✓	**Solution**

Final valid schedule: D1 = Morning, D2 = Afternoon, D3 = Night.

Other branch: D1 = A, D2 = M, D3 = N is also valid (the only other solution). D2 = N always fails because no later shift exists for D3, which triggers backtracking.

---

2. Robot Navigation in a 5×5 Grid (Best-First Search)

Assumed grid (cells written as (row, column), row 1 at the top; S = (1,1), G = (5,5), X = obstacle at (2,2), (3,3), (4,4)):

```
    c1 c2 c3 c4 c5
r1   S  .  .  .  .
r2   .  X  .  .  .
r3   .  .  X  .  .
r4   .  .  .  X  .
r5   .  .  .  .  G
```

Rules: moves Up, Down, Left, Right at cost 1; obstacles blocked. **Heuristic:** h(r, c) = |5 − r| + |5 − c|. **Greedy Best-First Search** orders the priority queue by h only; ties are broken by insertion order (neighbours generated Up, Down, Left, Right).

1047 - KAUSHIK

Node expansion and priority queue

Step	Expand (h)	Added to queue (h)	Priority queue after step (node: h)
1	(1,1) S (8)	(2,1) 7, (1,2) 7	(2,1):7, (1,2):7
2	(2,1) (7)	(3,1) 6; (2,2) is an obstacle	(3,1):6, (1,2):7
3	(3,1) (6)	(4,1) 5, (3,2) 5	(4,1):5, (3,2):5, (1,2):7
4	(4,1) (5)	(5,1) 4, (4,2) 4	(5,1):4, (4,2):4, (3,2):5, (1,2):7
5	(5,1) (4)	(5,2) 3	(5,2):3, (4,2):4, (3,2):5, (1,2):7
6	(5,2) (3)	(5,3) 2; (4,2) already queued	(5,3):2, (4,2):4, (3,2):5, (1,2):7
7	(5,3) (2)	(4,3) 3, (5,4) 1	(5,4):1, (4,3):3, (4,2):4, (3,2):5, (1,2):7
8	(5,4) (1)	(5,5) G 0; (4,4) is an obstacle	(5,5):0, (4,3):3, (4,2):4, (3,2):5, (1,2):7
9	(5,5) G (0)	Goal reached	–

192321047 - KAUSHIK

Path: (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5). **Cost = 8 moves.**

Optimality: greedy best-first is not optimal in general, because it ignores g(n). Here the cost 8 equals the Manhattan distance from S to G, which is a lower bound on any path, so this path is optimal. A* with f = g + h gives the same result and guarantees optimality when h is admissible.

---

3. Autonomous Rescue Robot (Online Search, Partially Observable)

a) How the constraints affect decisions

Constraint	Effect on decision-making
Moves Up/Down/Left/Right	At most 4 successors per cell; no diagonal shortcuts, so Manhattan distance is a safe lower bound (admissible heuristic)
Blocked cells	Some are unknown until sensed; a plan made earlier can become invalid, so the robot must replan
Sensing limited to adjacent cells	Partial observability: the robot keeps its own map (belief) and cannot compute a globally optimal path in advance; it must plan on what it knows and explore
Move cost 1, risky cell +2 (cost 3 to enter)	Step costs are not uniform, so BFS is unsuitable; a cost-sensitive search (UCS/A*) is needed. A risky cell is entered only if avoiding it adds more than two

192321047 - KAUSHIK

Constraint	Effect on decision-making
	extra moves
Minimise cost and stay safe	Blocked cells are hard constraints (never entered); risky cells are soft constraints (penalised in cost)
Update knowledge dynamically	Planning and acting are interleaved (online search); the map is updated after every move

b) Solution approach: online agent with A* replanning (D* Lite / LRTA*-style)

1. **Initialise:** the map contains only S and the known size of the grid. Unknown cells are assumed free and safe (optimistic free-space assumption).
2. **Sense:** read the four adjacent cells and update the map as free, blocked or risky.
3. **Plan:** run A* from the current cell to G on the known map with g = accumulated cost (1 per move, +2 for each risky cell entered) and h = Manhattan distance (admissible, since every move costs at least 1).
4. **Act:** execute only the first move of the plan.
5. **Repeat** from step 2. If a blocked or risky cell appears on the planned path, replan from the current position.
6. **No path on the known map:** go to the nearest unexplored frontier cell (online DFS-style exploration) and continue; if everything reachable is explored and G is unreachable, report failure.
7. **Stop** when the robot reaches G (survivor located).

192321047 - KAUSHIK

Mini-illustration: the robot at S plans straight toward G. At the next cell it senses a blocked cell ahead, adds it to the map, and A* immediately returns a path around it. Later it senses a risky cell on that path; the extra +2 makes a slightly longer safe path cheaper, so the plan switches to it.

c) Intelligent agent, rationality and environment type

PEAS: Performance measure = reach G, minimum total cost, avoid blocked/risky cells, safe. Environment = grid of rooms in a collapsed building. Actuators = drive wheels (Up/Down/Left/Right), rescue/alert module. Sensors = adjacent-cell sensors, position/odometry.

Agent type: a model-based, goal-based, utility-based (cost) agent. It maintains an internal map (state), has a goal (G) and prefers lower cost.

Rationality: the robot picks, at every step, the action that is expected to maximise performance given its percept history and current knowledge. Rationality is not omniscience: it exploits what it has sensed and explores when information is missing.

Environment properties: partially observable, single agent, deterministic (assuming moves succeed; aftershocks would make it stochastic), sequential, static between moves, discrete, and initially unknown.

d) Why this approach is most suitable

- Offline UCS or A* needs the complete map in advance, which this robot does not have; replanning solves that.
- BFS/DFS ignore the +2 risk cost, so they give unsafe or costly routes.
- A* with Manhattan distance focuses the search toward G (fewer expansions than UCS) and is optimal for the information currently known; replanning corrects the route as knowledge improves.

192321047 - KAUSHIK

- D* Lite reuses earlier search effort, so replanning after each new sensing is cheap.
- The method is complete when G is reachable and the grid is finite, and it avoids blocked cells by construction.

---

4. Map Colouring of South Indian States (CSP with Heuristics)

i. CSP formulation

- **Variables:** TN, KL, KA, AP, TG, GA, MH.
- **Domains:** {Red, Green, Blue} for each.
- **Constraints (inequality on each neighbouring pair):** TN≠KL, TN≠KA, TN≠AP, KL≠KA, KA≠AP, KA≠TG, KA≠GA, KA≠MH, AP≠TG, GA≠MH, MH≠TG.

Constraint graph (to be drawn): nodes are the seven states, an edge for each neighbouring pair. KA sits at the centre joined to all six others; the other six form a chain **KL – TN – AP – TG – MH – GA** around it.

State	TN	KL	KA	AP	TG	GA	MH
Degree	3	2	**6**	3	3	2	3

ii–iii. Backtracking with forward checking (Degree for the first variable, then MRV)

Step	Assignment (reason)	TN	KL	KA	AP	TG	GA	MH
0	Initial	RGB	RGB	RGB	RGB	RGB	RGB	RGB
1	KA =	GB	GB	**R**	GB	GB	GB	GB

1223121047 - KAUSHIK

Step	Assignment (reason)	TN	KL	KA	AP	TG	GA	MH
	R (highest degree, 6)							
2	TN = G (all tied at 2 values; tie broken by most unassigned neighbours)	**G**	B	R	B	GB	GB	GB
3	KL = B (MRV: 1 value)	G	**B**	R	B	GB	GB	GB
4	AP =	G	B	R	**B**	G	GB	GB

192321047 - KAUSHIK

Step	Assignment (reason)	TN	KL	KA	AP	TG	GA	MH
	B (MRV: 1 value); forward check removes B from TG							
5	TG = G (MRV: 1 value); removes G from MH	G	B	R	B	**G**	GB	B
6	MH = B (MR	G	B	R	B	G	G	**B**

192321047 - KAUSHIK

Step	Assignment (reason)	TN	KL	KA	AP	TG	GA	MH
	V: 1 value); removes B from GA							
7	GA = G (only value)	G	B	R	B	G	**G**	B

No domain became empty, so no backtracking was needed.

iv. Final colouring

KA = Red; TN = Green; KL = Blue; AP = Blue; TG = Green; MH = Blue; GA = Green.

Check: TN–KL G/B, TN–KA G/R, TN–AP G/B, KL–KA B/R, KA–AP R/B, KA–TG R/G, KA–GA R/G, KA–MH R/B, AP–TG B/G, GA–MH G/B, MH–TG B/G → all different.

How the heuristics helped: the degree heuristic colours KA first, the most constrained state, so forward checking prunes six domains at once. MRV then always picks a state with a single legal colour, so every assignment is forced and a conflict would show up immediately (an empty domain). The result is 7 assignments for 7 variables with zero backtracks.

192321047 - KAUSHIK

5. 4×4 Sudoku (Constraint Propagation with AC-3)

Cells are written (row, column). Given grid:

```
- 2 - -
3 - - 2
- - 4 -
4 - - 1
```

i. CSP definition

- **Variables:** the 16 cells X(r, c).
- **Domains:** {1, 2, 3, 4} for blank cells; a given cell has a one-value domain.
- **Constraints:** Alldiff on each of the 4 rows, 4 columns and 4 boxes of 2×2 (12 Alldiff constraints). Each Alldiff can be treated as binary ≠ arcs between every pair of cells in the same unit for AC-3.

ii. Initial domains (after removing values in the same row, column and box)

Cell	Values removed	Domain
(1,1)	2 (row), 3, 4 (column)	{1}
(1,3)	2 (row), 4 (column), 2 (box)	{1, 3}
(1,4)	2 (row), 2, 1 (column)	{3, 4}
(2,2)	3, 2 (row), 2 (column)	{1, 4}
(2,3)	3, 2 (row), 4 (column)	{1}
(3,1)	4 (row), 3, 4 (column)	{1, 2}

1923 1047 - KAUSH

Cell	Values removed	Domain
(3,2)	4 (row), 2 (column)	{1, 3}
(3,4)	4 (row), 2, 1 (column), 4, 1 (box)	{3}
(4,2)	4, 1 (row), 2 (column)	{3}
(4,3)	4, 1 (row), 4 (column)	{2, 3}

iii. AC-3 propagation

Singleton cells (1,1)=1, (2,3)=1, (3,4)=3 and (4,2)=3 are queued first; each revises the arcs to the cells in its row, column and box.

Revision (value removed)	Cell	Domain after
(1,1)=1 removes 1	(1,3)	{3}
(1,1)=1 removes 1	(2,2)	{4}
(1,1)=1 removes 1	(3,1)	{2}
(3,4)=3 removes 3	(1,4)	{4}
(3,4)=3 removes 3	(4,3)	{2}
(3,4)=3 or (4,2)=3 removes 3	(3,2)	{1}
(4,2)=3 removes 3	(4,3)	{2}

Every domain is now a singleton and no domain is empty, so the puzzle is arc-consistent and solved.

192321047 • KAUSHIK

iv. Completed grid

```
1 2 3 4
3 4 1 2
2 1 4 3
4 3 2 1
```

Every row, column and 2×2 box contains 1–4 exactly once. **Backtracking was not needed**: constraint propagation alone was enough.

---

6. Cryptarithmetic Puzzle (CSP with Global Constraints)

```
  T W O
+ T W O
-------
F O U R
```

i. Variables, domains, constraints

- **Variables:** T, W, O, F, U, R and carries C1, C2, C3.
- **Domains:** letters {0, …, 9} with T, F ∈ {1, …, 9} (leading letters are not zero); carries {0, 1}.
- **Constraints:** AllDiff(T, W, O, F, U, R) plus the four column equations below.

ii. Column constraints (right to left)

1. Units: O + O = R + 10·C1
2. Tens: W + W + C1 = U + 10·C2
3. Hundreds: T + T + C2 = O + 10·C3
4. Thousands: C3 = F

192321047 KAUSHIK

iii. Constraint hypergraph

Nodes: T, W, O, F, U, R, C1, C2, C3. Hyperedges (each drawn as a closed loop around its nodes):

Hyperedge	Nodes it connects
H0: AllDiff	T, W, O, F, U, R
H1: units column	O, R, C1
H2: tens column	W, U, C1, C2
H3: hundreds column	T, O, C2, C3
H4: thousands column	F, C3

C1 links H1 and H2, C2 links H2 and H3, C3 links H3 and H4, so the carries chain the columns together.

iv. Step-by-step solution

Deductions first: from H4, F = C3, and F ≠ 0 so **F = 1 = C3**. Then H3 gives 2T + C2 = O + 10, so T ≥ 5. H1 gives R = 2O − 10·C1, so R is even.

Step	Try	Constraint checks	Result
1	F = 1, C3 = 1	F ≠ 0 ✓	Keep
2	T = 5, C2 = 0	O = 2T + C2 − 10 = 0, so R = 0 = O (AllDiff ✗)	Reject
3	T = 5, C2 = 1	O = 1 = F (AllDiff ✗)	Reject

1923047 - KAUSHIK

Step	Try	Constraint checks	Result
4	T = 6, C2 = 0	O = 2, R = 4, C1 = 0; tens: 2W = U must be distinct and unused: W = 0 gives U = 0 (✗), W = 3 gives U = 6 = T (✗), other W values are used	Reject
5	T = 6, C2 = 1	O = 3, R = 6 = T (AllDiff ✗)	Reject
6	T = 7, C2 = 0	O = 4, R = 8, C1 = 0	Keep
7	W = 0	U = 2W = 0 = W (✗)	Reject
8	W = 2	U = 4 = O (✗)	Reject
9	W = 3	U = 6; digits T7 W3 O4 F1 U6 R8 are all different; tens: 3 + 3 + 0 = 6 ✓	**Solution**

192321047 - KAUSHIK

One valid assignment: T = 7, W = 3, O = 4, F = 1, U = 6, R = 8, C1 = 0, C2 = 0, C3 = 1.

Check: 734 + 734 = 1468 ✓. (Other solutions exist, for example 765 + 765 = 1530 and 938 + 938 = 1876; one is enough.)

---

7. Ambulance Routing During Floods (A* with Road Closure)

Road B–D is closed, so it is not used. Edges: S–A 3, S–B 2, A–C 4, B–C 1, C–D 2, C–E 3, D–G 2, E–G 4. Heuristic: h(S)=6, h(A)=6, h(B)=4, h(C)=4, h(D)=2, h(E)=3, h(G)=0.

i. A* trace (entries written node (g, h, f))

Step	Node expanded	Open list after expansion	Closed list
0	–	S(0,6,6)	–
1	S	B(2,4,6), A(3,6,9)	S
2	B	C(3,4,7), A(3,6,9)	S, B
3	C	D(5,2,7), A(3,6,9), E(6,3,9)	S, B, C
4	D	G(7,0,7), A(3,6,9), E(6,3,9)	S, B, C, D
5	G (goal)	–	S, B, C, D, G

192321047 - AUSHIK

Notes: in step 2, B–D is closed so D is not generated from B. In step 3, reaching A through C costs g = 7, worse than the existing g = 3, so it is ignored.

ii. Route found

S → B → C → D → G, total cost = 2 + 1 + 2 + 2 = 7 minutes.

iii. If B–D were open

Expanding B now also generates D(4, 2, 6): Open = {D(4,2,6), C(3,4,7), A(3,6,9)}. D is expanded next and generates G(6, 0, 6), which is expanded and returned. **Route S → B → D → G, cost = 2 + 2 + 2 = 6 minutes.** The closure adds 1 minute.

iv. Admissibility and consistency (flooded network)

True cost h* to G: D = 2, E = 4, C = 4 (via D), B = 5 (B→C→D→G), A = 8 (A→C→D→G), S = 7.

Node	S	A	B	C	D	E	G
h	6	6	4	4	2	3	0
h*	7	8	5	4	2	4	0
h ≤ h*?	✓	✓	✓	✓	✓	✓	✓

Admissible: yes. Consistency check h(n) ≤ c(n, n') + h(n') for every road, in both directions: S→A 6 ≤ 3+6; S→B 6 ≤ 2+4 (equality); A→C 6 ≤ 4+4; B→C 4 ≤ 1+4; C→D 4 ≤ 2+2 (equality); C→E 4 ≤ 3+3; D→G 2 ≤ 2+0 (equality); E→G 3 ≤ 4+0; and the reverse directions A→S, B→S, C→A, C→B, D→C, E→C also hold (for example C→D reversed: h(D)=2 ≤ 2+h(C)=6). **Consistent: yes.** So f never decreases along a path and A* returns the optimal route.

1923210417 - KAUSHIK

8. 4-Queens Problem (Local Search with Min-Conflicts)

i. CSP formulation

- **Variables:** Q1, Q2, Q3, Q4 (queen in each column; value = row number).
- **Domains:** {1, 2, 3, 4} for each.
- **Constraints:** for every pair $i < j$: $Qi \neq Qj$ (not in the same row) and $|Qi - Qj| \neq j - i$ (not on the same diagonal). Columns are distinct by construction.

ii. Initial board, rows (1, 3, 4, 2)

```
    c1 c2 c3 c4
 r1  Q  .  .  .
 r2  .  .  .  Q
 r3  .  Q  .  .
 r4  .  .  Q  .
```

Only Q2 (row 3, column 2) and Q3 (row 4, column 3) attack each other (diagonal). Conflicts: **Q1 = 0, Q2 = 1, Q3 = 1, Q4 = 0** (total 1 pair).

iii. Min-Conflicts

Move 1: the leftmost conflicted queen is Q2. Conflicts for each row of column 2, given Q1 at row 1, Q3 at row 4, Q4 at row 2:

Row for Q2	Row 1	Row 2	Row 3	Row 4
Conflicts	1 (row with Q1)	2 (diagonal with Q1, row with Q4)	1 (diagonal with Q3)	2 (row with Q3, diagonal with Q4)

Minimum is 1 at rows 1 and 3; the tie goes to the lowest row, so Q2 moves to **row 1**. State (1, 1, 4, 2):

```
    c1 c2 c3 c4
r1   Q  Q  .  .
r2   .  .  .  Q
r3   .  .  .  .
r4   .  .  Q  .
```

Conflicts: Q1 = 1, Q2 = 1 (same row), Q3 = 0, Q4 = 0.

Move 2: the leftmost conflicted queen is Q1. With Q2 at row 1, Q3 at row 4, Q4 at row 2:

Row for Q1	Row 1	Row 2	Row 3	Row 4
Conflicts	1 (row with Q2)	3 (diagonals with Q2, Q3; row with Q4)	**0**	1 (row with Q3)

Q1 moves to **row 3**. State (3, 1, 4, 2):

```
    c1 c2 c3 c4
r1   .  Q  .  .
r2   .  .  .  Q
r3   Q  .  .  .
r4   .  .  Q  .
```

Conflicts: Q1 = 0, Q2 = 0, Q3 = 0, Q4 = 0, total 0.

iv. Final solution

Rows (3, 1, 4, 2) for columns 1–4, found in 2 moves.

Why Min-Conflicts works for large n-queens: solutions are plentiful and densely spread through the state space, so a greedy repair of one queen at a time almost always makes progress. Each step looks at only one column and n rows, and conflict counts can be updated incrementally. The method needs only O(n) memory and the number of

1923 T. KAUSHIK

steps grows very slowly with n; it is reported to solve even a million queens in roughly 50 steps on average.

---

9. Missionaries and Cannibals (State-Space Search with Safety Constraints)

i. Formulation

- **State:** (M, C, B): missionaries and cannibals on the **left** bank and the boat side B ∈ {L, R}. The right bank holds 3 − M and 3 − C.
- **Initial state:** (3, 3, L). **Goal test:** (0, 0, R).
- **Actions:** the boat carries 1 or 2 people from its current bank: 1M, 2M, 1C, 2C or 1M+1C, subject to availability on that bank.
- **Transition model:** subtract the boatload from the bank the boat leaves and add it to the other bank, then flip B.
- **Path cost:** 1 per crossing.

ii. States pruned (unsafe)

A bank is unsafe if it has missionaries and more cannibals than missionaries.

- Left bank unsafe: (M, C) = (1,2), (1,3), (2,3).
- Right bank unsafe: (M, C) = (1,0), (2,0), (2,1).

Safe (M, C) pairs: (0,0), (0,1), (0,2), (0,3), (1,1), (2,2), (3,0), (3,1), (3,2), (3,3), i.e. 10 pairs, so at most 20 states with the boat position.

iii. BFS (states by depth, only safe unvisited states kept)

Depth	Move	State reached
0	start	(3,3,L)
1	2C, 1C or 1M+1C	(3,1,R), (3,2,R),

192321047 - KAUSHIK

Depth	Move	State reached
	across	(2,2,R); the moves 1M and 2M are pruned (unsafe left bank)
2	1C back from (3,1,R)	(3,2,L); other returns revisit earlier states or are unsafe
3	2C across	(3,0,R)
4	1C back	(3,1,L)
5	2M across	(1,1,R)
6	1M+1C back	(2,2,L)
7	2M across	(0,2,R)
8	1C back	(0,3,L)
9	2C across	(0,1,R)
10	1C back, or 1M back	(0,2,L), (1,1,L)
11	2C across from (0,2,L), or 1M+1C from (1,1,L)	**(0,0,R) goal**

192321047 - KAUSHIK

One solution path (11 crossings): (3,3,L) → 2C across → 1C back → 2C across → 1C back → 2M across → 1M+1C back → 2M across → 1C back → 2C across → 1C back → 2C across.

iv. Minimum crossings and why BFS guarantees it

Minimum = 11 crossings. BFS expands all states at depth k before any state at depth k + 1. Every crossing costs 1, so depth equals path cost, and the first time the goal is generated is at the smallest possible depth. The state space is finite (at most 20 states) and visited states are skipped, so the search terminates and never loops.

---

10. Sensorless Corridor Robot (Search with Partial Information)

i. Initial belief state and size of belief-state space

The robot could be in any of the four cells, so the **initial belief state is {1, 2, 3, 4}**. A belief state is any subset of the 4 cells, so there are $\mathbf{2^4 = 16}$ belief states (15 non-empty).

ii. Belief states for [Right, Right, Right]

After action	Belief state	Reasoning
(start)	{1, 2, 3, 4}	no information
Right	{2, 3, 4}	1→2, 2→3, 3→4, 4 stays in 4
Right	{3, 4}	2→3, 3→4, 4 stays
Right	{4}	3→4, 4 stays: the robot is certain it is in cell 4

1923217 - KAUSHIK

iii. No shorter sequence guarantees cell 4

The goal belief state is {4}, of size 1, starting from size 4. Each action is deterministic and moves every cell one step, so two different cells can end up in the same cell only at the wall: Right merges only cells 3 and 4 into 4, and Left merges only cells 1 and 2 into 1. So **every action reduces the size of the belief state by at most 1**. Going from size 4 to size 1 needs a reduction of 3, hence **at least 3 actions**. After any 2 actions the belief state has at least 2 cells, so the robot cannot be certain. Therefore [Right, Right, Right] is optimal.

iv. With a bump sensor (contingency plan)

```
Right
if bump then DONE (robot was in 4)
else Right
  if bump then DONE (robot is in 4)
  else Right
    DONE (robot is in 4 whether or not it bumped)
```

Reasoning: a bump on Right means the robot was already in cell 4. No bump on the first Right means the belief is {2, 3, 4}; no bump on the second means {3, 4}; after the third Right the robot must be in 4.

Starting cell	Actions with bump sensor	Actions sensorless
4	1 (bump on first Right)	3
3	2 (bump on second Right)	3
2	3	3
1	3	3

Comparison: the worst case is still 3 actions, but the contingency plan uses fewer actions on average (2.25 against 3) and tells the robot when it has arrived through the bump, so it can stop early. The sensorless plan is a fixed sequence that needs no feedback and is therefore robust to sensor failure, whereas the contingency plan is a conditional plan (a tree) that depends on the sensor.

192321047 - KAUSHIK